

Advanced Machine Learning

DATA 642

Lecture 3 — Quadratic Programming

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth

Quadratic Programming

Quadratic programming extends the capabilities of linear programming by allowing for optimization problems with quadratic objective functions and linear constraints.

- **Optimization of Nonlinear Models:** While linear programming is suitable for linear models, quadratic programming accommodates nonlinear models by enabling optimization with quadratic objective functions.
- **Support Vector Machines (SVMs):** Quadratic programming plays a crucial role in the optimization of support vector machines (SVMs). SVMs involve the minimization of a quadratic objective function subject to linear constraints, making quadratic programming an essential tool for training SVM models.
- **Regularization and Penalization:** In various machine learning applications, regularization and penalization terms are incorporated into the objective function to prevent overfitting and encourage simpler models. Quadratic programming facilitates the optimization of such objectives, leading to improved generalization performance and model interpretability.
- **Complex Constraint Handling:** Quadratic programming can handle complex constraints more effectively than linear programming, allowing for the incorporation of nonlinear constraints or additional regularization terms in the optimization process.

The mathematical setup

Consider the case of a convex quadratic objective function, where the constraints are affine, i.e.,

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \frac{1}{2} \boldsymbol{\theta}^T Q \boldsymbol{\theta} + \boldsymbol{c}^T \boldsymbol{\theta} \quad \text{subject to} \quad A \boldsymbol{\theta} \leq \boldsymbol{b} \quad (1)$$

where $A \in \mathbb{R}^{m \times d}$, $\boldsymbol{b} \in \mathbb{R}^m$, and $\boldsymbol{c} \in \mathbb{R}^d$. The square symmetric matrix $Q \in \mathbb{R}^{d \times d}$ is positive definite, and therefore the objective function is **convex**. This is known as a quadratic program. Observe that it has d variables and m linear constraints.

The Lagrangian is given by

$$\mathcal{L}(\boldsymbol{\theta}; \boldsymbol{\lambda}) = \frac{1}{2} \boldsymbol{\theta}^T Q \boldsymbol{\theta} + \mathbf{c}^T \boldsymbol{\theta} + \boldsymbol{\lambda}^T (A \boldsymbol{\theta} - b) \quad (2)$$

Taking the derivative of $\mathcal{L}(\boldsymbol{\theta}; \boldsymbol{\lambda})$ with respect to $\boldsymbol{\theta}$ and setting it to zero gives

$$Q \boldsymbol{\theta} + (\mathbf{c} + A^T \boldsymbol{\lambda}) = 0 \quad (3)$$

Assuming that Q is invertible, we get

$$\boldsymbol{\theta} = -Q^{-1}(\mathbf{c} + A^T \boldsymbol{\lambda}) \quad (4)$$

Substituting $\boldsymbol{\theta} = -Q^{-1}(\mathbf{c} + A^T \boldsymbol{\lambda})$ into the primal Lagrangian $\mathcal{L}(\boldsymbol{\theta}; \boldsymbol{\lambda})$, we get the dual Lagrangian

$$D(\boldsymbol{\lambda}) = -\frac{1}{2}(\mathbf{c} + A^T \boldsymbol{\lambda})^T Q^{-1}(\mathbf{c} + A^T \boldsymbol{\lambda}) - \boldsymbol{\lambda}^T b \quad (5)$$

Therefore, the dual optimization problem is given by

$$\max_{\boldsymbol{\lambda} \in \mathbb{R}^m} -\frac{1}{2}(\mathbf{c} + A^T \boldsymbol{\lambda})^T A Q^{-1}(\mathbf{c} + A^T \boldsymbol{\lambda}) - \boldsymbol{\lambda}^T b \quad (6)$$

subject to $\boldsymbol{\lambda} \geq 0$.

Application: Support Vector Machines (SVMs)

The optimization problem associated with SVMs can be formulated as a quadratic program because the optimization problem involves minimizing a quadratic objective function subject to linear constraints.

$$\underset{\boldsymbol{\theta} \in \mathbb{R}^l}{\text{minimize}} \quad \frac{1}{2} |\boldsymbol{\theta}|^2 + C \sum_{n=1}^N \xi_n \quad \text{subject to} \quad y_n (\mathbf{x}_n^T \boldsymbol{\theta} + b) \geq 1 - \xi_n, \xi_n \geq 0 \quad (7)$$

In this formulation: $\boldsymbol{\theta}$ represents the hyperplane parameters (weights). C is a regularization parameter that controls the trade-off between maximizing the margin and minimizing the classification errors. $\mathbf{x}_n$ and y_n are the input features and labels of the training data, respectively. ξ_n are slack variables that allow for some misclassification, penalized by the regularization term.